

Model.py Contains the `Model` class

BarView.py Contains the `BarView` class that displays the bar chart

Bin.py Contains the `Bin` class that draws a single bar in the bar chart

PieView.py Contains the `PieView` class that displays the pie chart

TextView.py Contains the `TextView` class that displays the text view

Constants.py Contains constants that are shared by multiple modules

The main program instantiates a `Controller` object and runs the main loop. Code in the main loop forwards all events (other than the `pygame.QUIT` event) to the controller to handle.

The Controller

The controller is the supervisor of the whole program. It starts by instantiating the `Model` object. It then instantiates one of each of the different view objects: `BarView`, `PieView`, and `TextView`. Here is the startup code in the `__init__()` method of the `Controller` class:

```
# Instantiate the model
self.oModel = Model()
# Instantiate different view objects
self.oBarView = BarView(self.window, self.oModel)
self.oPieView = PieView(self.window, self.oModel)
self.oTextView = TextView(self.window, self.oModel)
```

When the `Controller` object instantiates these `View` objects, it passes in the `Model` object so each `View` object can request information directly from the model. Different implementations of the MVC pattern might handle communication between these three elements differently; for example, a controller could act as an intermediary, requesting data from the model and forwarding it on to the current view, rather than allowing the model and view to communicate directly.

The controller draws and reacts to everything outside the black rectangle in the window, including the title, the image of the dice, and the radio buttons. It draws the Quit and Roll Dice buttons and reacts when they're clicked, and it handles any changes the user makes to the number of rolls.